Python OrderedDict and Ordered Set: one-page summary

n = items, k = size of the other collection, avg = average case for hashing. Python has no built-in OrderedSet.

OrderedDict (collections)

Hash map plus doubly linked list. Remembers insertion order and reorders in O(1). Plain dict also keeps order (3.7+) but lacks the first four rows.

Operation	Time	Note
`od.move_to_end(k, last=True)`	**O(1)**	To end, or front with last=False. KeyError if missing.
`od.popitem(last=True)`	**O(1)**	LIFO. last=False is FIFO (oldest). KeyError if empty.
`reversed(od)`	O(n)	Newest to oldest.
`od1 == od2`	O(n)	Order-sensitive. Against a dict, order ignored.
`od[k], od.get(k,d), k in od, od.setdefault(k,d), del od[k], od.pop(k,d)`	O(1) avg	New key goes to end. Updating a key keeps its position.
`len(od), next(iter(od)), next(reversed(od))`	O(1)	Size, oldest key, newest key.
`od.keys(), .values(), .items()`	O(1) view, O(n) loop	Insertion order. Mutating while looping: RuntimeError.
`od.update(o), od | o (3.9+)`	O(k), O(n+k)	New keys appended, old keep position.
`od.copy(), od.clear(), OrderedDict.fromkeys(it), for k in od`	O(n)	
`list(od)[i]`	O(n)	No index access (main weakness).

Ordered set, option A: plain dict (s = {})

Keys are the items, values are None. Most common in interviews.

Operation	Time	Note
`s[x]=None (add), s.pop(x,None) (discard), del s[x] (remove), x in s`	O(1) avg	Re-adding keeps position. del raises KeyError.
`len(s), s.popitem()`	O(1)	popitem removes the newest.
`next(iter(s)); del s[x]`	O(1) typical	Oldest item. Repeated front removal degrades: 0.078 s at 20k vs 1.5 s at 80k items. Use B.
`{**a,**b}, a | b`	O(n+k)	Union.
`{x:None for x in a if x in b}`	O(len a)	Intersection. Use 'not in' for difference.
`list(dict.fromkeys(items))`	O(n)	Dedupe, keep first-seen order.
`list(s).index(x)`	O(n)	No positions.

Option B: OrderedDict as a set

All of A, plus two O(1) moves a plain dict cannot do cleanly: `s.popitem(last=False)` removes the oldest, `s.move_to_end(x)` makes x the newest. Used in LRU (146) and LFU (460).

Option C: ordered-set package (third-party)

Usually a list plus index map, so: add, update, `in`, len, `s[i]`, `index(x)`, `pop()` (last) are O(1). `discard/remove` is O(n) because later indexes shift. Union, intersection, difference and subset tests are O(n+k). Not allowed on LeetCode. Costs are from its usual design, not tested here.

When to use each

Need	Use	Why
Key to value, order irrelevant	dict	
LRU cache / reorder by recency	OrderedDict	O(1) move_to_end + popitem(last=False)
LFU bucket (oldest key per freq)	OrderedDict as set	O(1) add, delete any, pop oldest
Dedupe, keep first-seen order	dict.fromkeys(x)	One line, O(n)
FIFO queue with cancel	OrderedDict as set	O(1) remove any + pop oldest
Order-sensitive dict equality	OrderedDict	plain dict == ignores order
Membership only	set	Fastest, real set algebra
Index by position often	list / ordered-set pkg	OrderedDict has no O(1) index
Push/pop at both ends	deque	O(1) both ends
Always need the smallest	heapq	O(log n) push/pop, O(1) peek
Sorted by value	sortedcontainers / bisect	Insertion order is not sorted order

Pitfalls

- Insertion order is not sorted order. Use SortedSet for sorted.
- Updating an existing key never moves it. Call move_to_end(k) to make it newest.
- Do not add or remove keys while looping. Loop over list(od) instead.
- move_to_end and popitem raise KeyError on a missing key or empty dict.
- Compound operations are not thread-safe. Guard them with a Lock.

One-liner: OrderedDict is a dict plus a linked list, so it can reorder and pop from either end in O(1). An ordered set is the same thing with None values.